Programming in C — Problem Set

Wan-Lei Zhao
Xiamen University

2024-5-19

Abstract

The exercises are organized in the order that is in line with the order of lecture chapters. For the exercises in one section, they are organized from the "easy" to "hard" order. The GCC compiler and Linux environment are recommended. For Windows users, you can try MingW compiler[1] and Visual Studio code editor.

My happiness with computer science starts from the learning of programming in C on my own in year 1999. From that time, I started to figure out how the computer works internally. I wish the same happiness is experienced by every user of this material.

Contents

I Branch clause 2
II Loop 2
III Function 3
IV Array 4
V Pointers 5
VI File Operations 5
VII Bitwise Operations 5
VIII Make and CMake 6

[1] https://www.mingw-w64.org/

I Branch clause

1. Given a user input of three integers **year**, **month**, and **day**, print out this day is which day of the year. Do it with switch clause.

II Loop

1. Find out all *Armstrong* numbers that are less than 100,000. *Armstrong* number: Given an n-digit number, the summation on the 'n' power of each digit number equals to the number.

```
Example: 1 2 3 4 5 6 7 8 9 153 370 371 407
```

2. Convert a decimal float number into binary loop until the residue is lower than 0.005. Print out the binary codes on the screen.

3. Print out the two pyramids of characters in the following shape:

```
A          B
AA        BB
AAA      BBB
AAAA    BBBB
AAAAABBBBB
```

4. Implement e^x, the x is specified by the user. Please only consider the term that is larger than 1e-6.

5. Print out the first n terms of *Fibonacci* series. 'n' is specified by the user.

$$F_n = \begin{cases} 0 & n = 0 \\ 1 & n = 1 \\ F_{n-1} + F_{n-2} & n > 1 \end{cases} \tag{1}$$

6. Given a character 'a' - 'z' or 'A' - 'Z', if it is in the lower case, convert it to the upper case, if it is in the upper case, convert it to the lower case.

7. Given two positive integers, work out the least common multiple of the two integers.

8. Find out all four-digits number that the sum of its first and the fourth digits equals to the sum of its second and its third digits. Print out all these numbers, each number takes up 7 positions, and separated by one blank. Five numbers at most should be printed on each line. The numbers on each column are left-aligned.

9. Find out all *Palindrome* numbers ($< 10,000$), print 8 numbers at most on each line.

10. Find out all numbers ($< 10,000$) that they are both squares and *Palindrome* numbers. Print 8 numbers at most on each line.

11. Accept an integer input 'n' from user, then output a 'n' rows of characters in a pyramid shape. For instance, when the user input 4, your codes print out the pyramid shape as:

```
   A
  BBB
 CCCCC
DDDDDDD
```

12. Accept an integer input 'n' from user, then the print out a 'n' rows of characters in a pyramid shape on the screen. For instance, when the user input 4, your codes print the pyramid shape on the screen like:

```
   A
  BCD
 EFGHI
JKLMNOP
```

13. Accept an integer input from user, then print out the integer number with the digits in reversed order. For instance, the input is "879", the output is "978". The input is "810", the output is "18".

14. Given integer range $[a, b]$ input by the user, print out the prime numbers that satisfy certain conditions. For the result prime number, it should satisfy the following condition. Namely, the digit in the higher position should be larger than the digit in the lower position. For instance, number "631" is a valid number, while prime number "113" is not. When user input "100 10000", one should print out all the valid prime numbers in this range. The valid output should be formatted with no more than 15 numbers on each line. One number occupies four digits position.

15. Find out all integers that their sum of all digits is 25. Print 10 numbers on each line. One number takes up 8 digit positions.

III Function

1. Please calculate $\sin(x)$ by its progression form. Given any float number input x, output its $\sin(x)$ value.

2. Please calculate $\cos(x)$ by its progression form. Given any float number input x, output its $\cos(x)$ value.

3. Define a function to judge whether an input integer is a prime number. In the `main()` function, print out all the prime numbers in the range of $[2, 1000]$.

4. List out all the *Fibonacci* numbers that are less than and equal to 10,000. When you print the number, each number takes up 5 digits. The number should be also divisible by integer

'5'.

$$F_n = \begin{cases} 0 & n = 0 \\ 1 & n = 1 \\ F_{n-1} + F_{n-2} & n > 1 \end{cases} \tag{2}$$

5. Define two functions to check whether one number is prime and whether there is a duplicate digit in the number respectively. Call and test them in "main()".

6. Define a function to judge whether a three-digit integer is a narcissistic number. Call this function from the `main()` function to check the numbers from 100 to 999, print out all the narcissistic numbers in this range.

IV Array

1. Given a numeric string in float number form "-321.12", please work out a function to convert it into a float number without calling "atof()" C library.

2. Given a user input of three integers year, month, day, output this day is which day of the year. Do it with 1D array.

3. Given a numeric string such as "321" that in pure positive integer form, work out a function to convert it into an integer number without calling "atoi()" C library.

4. Transpose a 2D matrix that is kept in a 1D array. Print out the matrix in rows and columns both before and after the transpose.

5. Swap the minimum with the maximum number of an array and print out the array before and after the swapping.

6. Given 10 children stand in a circle, they count off from '1'. Each time, the one who counts $n = 3$ gets out from the circle. The ones in the circle continue this counting until no children are left. Print out the order that the children get out from the circle. The user can specify 'n'.

7. Given matrices A of 3×4 and B of 4×3, implement the multiplication between two matrices, and print out the resulting matrix.

8. Given two sorted arrays, merge them into a new one while keeping the resulting array sorted.

9. Given a sorted array, insert three numbers (unordered) from another array into the sorted array one by one.

10. Given a sentence kept by a string, where words are separated by blanks, remove extra blanks. There should be only one blank between words and no blank at the end.

11. Calculate the frequency of characters in a string (case-insensitive). List the frequency of each character that appears at least once.

12. Given a decimal integer, print out its hexadecimal number.

13. Implement a function to convert an integer into a string. For example, input -312, print out "-312".

14. Print out the factorial of n, namely $n!$, where n is an integer in the range of $[80, 110]$.

V Pointers

1. Build a single-directed list structure, no longer than 26. The length is specified by the user. Values are uppercase characters (e.g., $A \rightarrow B \rightarrow C \rightarrow D \rightarrow NIL$). Reverse the list without the support of extra list or array and print the reversed list out.

2. Given a root of a binary tree, define a recursive function to calculate the depth of the tree.

3. Define a recursive function to calculate the depth of an M-branch tree.

4. Define a function pointer pointing to functions such as sin(x), x^2, e^x that are defined by yourself as well. It is allowed the user to choose based on which to calculate its integral between range $[a, b]$, the parameters a and b are also specified by the user. The code should print out the integral value for the specified function and integral range.

VI File Operations

1. Allow the user to input the 10 student records that will be kept in an array of structure. Thereafter, save the 10 records to file "std_table.db" in binary form. Finally, load the records from the file into an array again and display them on the screen as a table.

```c
typedef struct {
 char stdName[64];
 char stdNumb[16];
 unsigned age;
} StdType;
```

2. Try the above problem, while save the records into a text file "std_table.txt".

VII Bitwise Operations

1. Given a number, check whether it is even or odd by bitwise operation.

2. Count the number of ones "1" for a user input 32-bit integer and print it out.

3. Given a non-negative number n. The problem is to toggle the last m bits in the binary representation of n. A toggle operation flips a bit from 0 to 1 and a bit from 1 to 0.

4. Given two non-negative integers a and b. The problem is to check if one of the two numbers is 1's complement of the other.

5. Given a string s representing a non-negative decimal number, determine whether the number is divisible by 8. Return true if it is divisible by 8, otherwise return false.

6. Given a number, the task is to set all odd bits of a number. Positions of bits are counted from LSB (least significant bit) to MSB (Most significant bit). Position of LSB is considered as 1.

```
Input : 20
Output : 21
Explanation : Binary representation of 20 is 10100. Setting all odd bits make
    the number 10101 which is binary representation of 21.
```

```
Input : 10
Output : 15
```

7. Check if a number is multiple of 9 using bitwise operators.

VIII Make and CMake

1. Declare two functions int func1(int a) and int func2(int b) and save them in "func1.h" and "func2.h" respectively, implement them in files "func1.c" and "func2.c". Therefore call func2() in another code file "proj1.c" where "main()" is defined. Save all these files under one folder. Please compose a "Makefile" manually to compile this project.

```c
int func1(int a);
```

Listing 1: func1.h

```c
#include <stdio.h>

int func1(int a)
{
    printf("a is %d\n", d);
    return a;
}
```

Listing 2: func1.c

```c
int func2(int b);
```

Listing 3: func2.h

```c
#include <stdio.h>
#include "func1.h"

int func2(int b)
{
    func1(a+2);
    printf("a is %d\n", d);
    return a;
}
```

Listing 4: func2.c

```c
#include <stdio.h>
#include "func2.h"

int main()
{
    printf("I am in main()\n");
    func2(5);
    return 0;
}
```

Listing 5: proj1.c

2. Try the above problem by composing a "CMakeLists.txt" file manually to compile the project.